

RESEARCHLY AI · R26-IT-116

Module 1

Research Integrity & Compliance

Citation intelligence, research-gap analysis, proposal generation,
plagiarism detection, and mind-map construction —
architecture, ML methodology, and calculation logic

Module owner: K. D. T. Kariyawasam

Service: `services/module1-integrity` (FastAPI, port 8001)

Frontend: `apps/web/src/app/(dashboard)/citations/*`

Document generated: 27 August 2026

Contents

1. Overview	How the module works
2. System Architecture & Data Flow	Frontend → Gateway → ML service → DB
3. Feature A — Citation Parser & Formatter	Regex engine, no ML
4. Feature B — Research Gap Analysis	SBERT retrieval + intelligence layer
5. Feature C — Proposal Generator	Retrieval-augmented composition
6. Feature D — Plagiarism Checker	SBERT + pgvector
7. Feature E — Mind Map Builder	Gemini + local fallback
8. Planned vs. Production Models	Spec deep-learning models vs. shipped code
9. Endpoint Reference	Full API surface

1 • Overview

Module 1 is the **research-integrity toolkit** of the platform. It helps a student move from a raw idea to a defensible, correctly-cited research proposal by automating five tasks that would otherwise be manual and error-prone:

- 1. **Citation parsing & formatting** — turn a messy pasted reference into structured fields and render it in APA/IEEE.
- 2. **Research-gap analysis** — given a topic, surface concrete, evidenced gaps in the existing SLIIT literature.
- 3. **Proposal generation** — draft a problem statement, objectives, methodology and expected outcomes grounded in real prior work.
- 4. **Plagiarism checking** — flag passages of a draft that are semantically too close to papers already in the corpus.
- 5. **Mind-map building** — turn free text into a hierarchical concept graph for visual brainstorming.

All five features live in one FastAPI microservice (`services/module1-integrity` , port `8001`) and are consumed by five corresponding pages under `apps/web/src/app/(dashboard)/citations/` (`parser` , `gaps` , `proposal` , `plagiarism` , `mindmap`).

Design principle used throughout Module 1: every feature that needs semantic understanding is built on **SBERT sentence embeddings** over a corpus of **3,858–4,219 real SLIIT undergraduate research papers** (scraped by Module 3 and shared across modules). This keeps every result *grounded* in real prior work — gaps, proposals and similar-paper suggestions all cite an actual paper, author and year, rather than being invented by a language model.

2 • System Architecture & Data Flow

Every request follows the same path through the stack:

```
Next.js page (citations/*) | fetch() with Supabase JWT ▼ Express API Gateway (apps/api-gateway, port 3001) |
requireAuth → mlRateLimiter → callMLService(1, path, method, body) ▼ FastAPI Module 1 service (services/module1-
integrity, port 8001) | router → service layer (citation_engine / gap_analyzer / gap_intelligence / |
proposal_retriever / plagiarism / mindmap) ▼ Supabase Postgres + pgvector (research_papers.embedding, 768-dim) or
local pickled SBERT index (models/trained_gap_analyzer, trained_proposal_retriever) or Gemini 2.5 Flash API
(fallback / concept extraction)
```

The gateway route file is `apps/api-gateway/src/routes/module1.routes.ts` . It applies `requireAuth` (Supabase JWT check) and `mlRateLimiter` to every route, then forwards the request body verbatim to the FastAPI service via `callMLService(1, path, method, body)` (`apps/api-gateway/src/services/mlProxy.service.ts`). PDF uploads (`/gaps/analyze-pdf`) are the one exception — they are re-packaged as `multipart/form-data` and proxied directly with `axios` because Multer already holds the file in memory on the gateway side. Module 1 calls get a 60-second proxy timeout.

Every ML-capable sub-feature follows the same **three-tier fallback** pattern so the product never hard-fails:

Tier	Source	Used by
1	Locally-trained SBERT index (pickled <code>.pkl</code> in <code>models/trained_*</code>)	Gap analysis, proposal generation, similar-papers, plagiarism
2	Gemini 2.5 Flash (<code>services/shared/gemini_client.py</code>)	Proposal generation fallback, mind-map concept extraction
3	Deterministic local heuristic (regex / term-frequency)	Citation parsing (always), mind-map fallback, proposal last-resort template

3 · Feature A — Citation Parser & Formatter

No ML — deterministic engine

What it does

The Citation Parser page lets a student paste a raw reference string (in any rough shape) and get back: structured fields (authors, title, year, journal/conference/publisher, volume, issue, pages, DOI), a fully formatted APA and IEEE citation, in-text citation forms, and a list of missing-field warnings. It also looks up a DOI or title against CrossRef, and can format a whole reference list in either style.

Why no ML here

Citation strings follow well-defined bibliographic conventions (author-year-title-venue ordering, ISO DOI format, etc.). A carefully-tuned regex/heuristic pipeline is both more accurate and far cheaper than an ML model for this structured-extraction task, and — critically — it is *fully deterministic and auditable*, which matters for something a student will submit in an actual thesis.

How it is implemented

The engine lives in `app/services/citation_engine.py` (self-contained, no external calls) and is invoked from `app/routers/citation.py`. The pipeline for `POST /citations/parse`:

1. `detect_source_type()` — regex hints classify the reference as *journal / conference / book / website* (URL without DOI/journal hint → website; "Proc./Conf./Workshop" → conference; "Journal/Trans./Review" → journal; known publisher names → book; DOI present with no other hint → journal by default).
2. DOI (`10\.\d{4,9}/...`), URL, publisher, volume, issue and page-range are each extracted by dedicated regexes.
3. **Year extraction:** every 4-digit token between 1800–2099 is collected; the latest one is taken as the publication year (handles "in press since 2019, published 2021" style strings).
4. **Author/title split:** if a year was found, the text is split at the year marker — everything before is parsed as the author list (`_extract_authors` , pairing "Last, Initials" tokens via `_smart_split_authors`), everything after up to the first sentence boundary is the title, and the remainder is attached as the venue (`_attach_venue`) depending on source type. When no year is found, a coarser first/second-sentence fallback (`_fallback_split`) is used.
5. A **confidence score** (0–1) is computed as a weighted sum over which fields were successfully extracted — see formula below.

$$\text{confidence} = 0.25 \cdot [\text{authors found}] + 0.25 \cdot [\text{title found}] + 0.15 \cdot [\text{year found}] + 0.10 \cdot [\text{venue found, field depends on source_type}] + 0.10 \cdot [\text{doi found}] + (0.05 \cdot [\text{url found}] \text{ only for websites})$$

Formatting (`format_apa / format_ieee`) then renders the structured record per style: APA orders *Authors. (Year). Title. Journal, vol(issue), pages. DOI* with sentence-case titles (`_sentence_case` lower-cases everything except the first word, ALL-CAPS acronyms, and the word after a colon); IEEE orders *Initials. Last, "Title," in Venue, pp. xx, Year, doi:* with quoted titles. In-text citations follow the same duality: APA gives (Last, Year) / (Last & Second, Year) / (Last et al., Year) depending on author count, IEEE gives a bracketed running index [n]. `POST /citations/reference-list` renders a whole list — IEEE preserves input order (numbered), APA re-sorts alphabetically by first-author surname then year.

DOI / title lookup

`app/services/crossref_lookup.py` calls the free CrossRef REST API (`api.crossref.org/works/{doi}` or `/works?query.bibliographic=`) and normalises its JSON shape into the same internal `parsed` record used everywhere else, so a DOI lookup goes through the identical formatter/warning pipeline as a manually-typed citation.

Similar-papers (the one SBERT touchpoint in this feature)

`POST /citations/similar-papers` reuses the same SBERT index built for the proposal retriever (`models/trained_proposal_retriever/proposal_index.pkl` , 3,858 SLIIT papers) via `app/services/sliit_examples.py`: the query is embedded, cosine-scored against every paper's pre-computed embedding (`_INDEX["embeddings"] @ qvec`), and the top-K rows are returned with a 280-character abstract excerpt. This lets a student check "has someone already written this exact paper?" directly from the citation-parser screen.

4 · Feature B — Research Gap Analysis

SBERT retrieval

rule-based intelligence layer

What it does

Given a free-text topic (or an uploaded/pasted full paper), the Gaps page returns: a ranked list of concrete research gaps (each grounded in a real supporting SLIIT paper), a classification of each gap (future-work / methodology / dataset / domain / performance), year-over-year trend analysis with a plain-English interpretation, cross-domain opportunity suggestions, over-researched ("saturated") subtopics to avoid duplicating, and 3–5 auto-generated research-direction recommendations with problem statements. A downloadable, self-contained HTML report (printable to PDF) is also available.

Why SBERT (and why a two-layer design)

"Research gap" is not a field in any database — it is a *semantic property* of the corpus: a sentence like *"further work is needed on X in the Sri Lankan context"* is a gap regardless of exact wording. SBERT sentence embeddings let the system match a student's topic against a pre-built index of such gap sentences purely by meaning, without keyword overlap. The retrieval step alone (`gap_analyzer.py`) gives similarity-ranked candidates; a second **intelligence layer** (`gap_intelligence.py`) is rule-based on top of that — it classifies, scores, explains, and synthesises trends/recommendations without requiring its own trained model, keeping the pipeline fast, deterministic and cheap to run on CPU.

How it is implemented — retrieval (`app/services/gap_analyzer.py`)

1. Lazy-loads `models/trained_gap_analyzer/gap_index.pkl` — a pickle of pre-extracted gap sentences (`gap_text` , `gap_type` , `year` , `paper_id` , ...) plus their SBERT embeddings — and the SLIIT-fine-tuned SBERT encoder from `models/sbert_plagiarism/` (falls back to base `all-MiniLM-L6-v2` if fine-tuned weights are absent).
2. The query topic is encoded and normalised; since both query and index vectors are L2-normalised, cosine similarity reduces to a single matrix multiply: `sims = embeddings @ query_vec` .
3. The top `max(40, top_k*5)` candidates are pulled, filtered by `min_similarity` , and each is scored on three axes (formulas below), then **near-duplicate clustered**: a greedy pass keeps a gap only if its cosine similarity to every already-kept gap is below `DEDUP_THRESHOLD = 0.78` , so the same underlying gap surfaced by several papers isn't shown three times.

```
gap_score = min(1, similarity * type_weight) type_weight ∈ {0.95 ... 1.10}, e.g. research_gap=1.10, limitation=0.95
recency = min(0.95, 0.20 + (current_year - paper_year)/12) — older supporting paper ⇒ "staler" gap novelty =
min(1, 0.5*(1 - (current_year - paper_year)/10) + 0.5*similarity) — recent AND relevant ⇒ high novelty
```

How it is implemented — intelligence layer (`app/services/gap_intelligence.py`)

- **Classification** — five regex pattern families (`CLASS_PATTERNS`) map each gap sentence to one of *future_work* / *methodology* / *dataset* / *performance* / *domain*, falling back to the corpus's own `gap_type` tag, else *general*.
- **Multi-dimensional ranking score** (used to sort the final top-K instead of raw similarity):

```
multi_dim_score = 0.45*(similarity * type_weight) + 0.20*(1 - recency) + 0.20*novelty + 0.15*similarity
```

- **Trend analysis** (`compute_trends`) — buckets supporting papers by year, then fits a least-squares linear slope over the last five years (`numpy.polyfit`). The slope is normalised by the mean count; $>+15\%/yr \rightarrow$ "rising", $<-15\%/yr \rightarrow$ "declining", otherwise "stable" — each mapped to a canned interpretation sentence.
- **Cross-domain opportunities** (`cross_domain_opportunities`) — ten hand-authored domain regexes (AI/ML, Healthcare, Agriculture, Education, Finance, Tourism, IoT, Cybersecurity, Sustainability, Sri Lanka/Regional). The query's primary domain is matched against every other domain; a pair with an intersection ratio < 0.20 of the primary domain's paper count is flagged as an opportunity, scored as `1 - ratio` .
- **Saturation detection** (`detect_saturation`) — simple term-frequency count across matched papers' titles/topics (stop-words removed); any term appearing in ≥ 4 papers is flagged as "well-explored — differentiate your contribution".
- **Recommendation synthesis** (`recommend_directions`) — one templated research-direction title/rationale/problem-statement per gap category present (from `_gap_to_title` / `_gap_to_rationale` / `_gap_to_problem` template dictionaries), plus up to two more from the strongest cross-domain pairs — capped at 5 total.

PDF / full-paper input path

`POST /gaps/analyze-pdf` and `POST /gaps/analyze-full-paper` extract text (`pdf_extractor.py` , `pypdf` → `pdfplumber` fallback), split it into abstract / methodology / conclusion sections via `split_paper_sections` , and build a focused query string (abstract first 1500 chars + methodology 1000 chars + conclusion 800 chars) before running the same `gap_intelligence.analyze()` pipeline — so an uploaded paper is analysed against the corpus using its most gap-relevant sections rather than the whole raw text.

5 • Feature C — Proposal Generator

Retrieval-augmented composition

Gemini fallback

What it does

Given a topic (and optional domain), the Proposal page returns a structured draft: a problem statement, 3–5 objectives, a methodology paragraph, expected outcomes, and the list of SLIIT papers the draft was grounded in (with similarity scores).

Why retrieval instead of a generative LLM

The spec originally called for a fine-tuned RAG+LoRA causal LLM (Mistral-7B/Llama-2 — see `app/models/proposal_generator.py`, still present as a wrapper class). In production, the live endpoint uses a lighter **retrieval + templating** approach instead (`app/services/proposal_retriever.py`): find the top-K most similar real SLIIT proposals by SBERT cosine similarity, then stitch a new proposal by combining the topic with sentence fragments pulled from those exemplars (their problem-statement/abstract, methodology snippets, and objective verbs). This trades a small amount of generative flexibility for two things that matter more here: *zero hallucination* (every claim is traceable to a cited SLIIT paper and author) and *zero GPU/latency cost* (a CPU matmul instead of an LLM forward pass), while a Gemini-based generative fallback still exists for genuinely novel topics with no close precedent in the corpus.

How it is implemented

1. Loads `models/trained_proposal_retriever/proposal_index.pkl` (SLIIT paper records + embeddings) and the same SLIIT-tuned SBERT encoder.
2. The query `"{topic} ({domain})"` is embedded and cosine-scored against every paper (`embeddings @ qvec`); the top-K exemplars are kept with their similarity.
3. **Problem statement** (`_compose_problem_statement`) — quotes the top exemplar's problem statement/abstract (truncated to 350 chars) attributed to its authors and year, wrapped in a fixed template sentence.
4. **Objectives** (`_compose_objectives`) — starts from two generic SLR-style objectives, then regex-extracts 1–2 action sentences per exemplar (pattern: `\b(propose|develop|design|evaluate|implement|analyze|assess|construct)\b...`) up to 5 total, de-duplicated by an 80-character key.
5. **Methodology** (`_compose_methodology`) — concatenates methodology-text snippets (truncated to 220 chars, up to 3 exemplars) into a fixed "mixed-methods" template paragraph.
6. **Expected outcomes** (`_compose_expected_outcomes`) — three fixed contribution sentences plus one noting how many prior SLIIT studies the work directly extends.

If the local index is unavailable, the router falls back to a Gemini prompt asking for the same JSON shape (problem_statement/objectives/methodology/expected_outcomes); if even that fails, a final deterministic template fills in generic sentences so the endpoint never returns an error to the UI.

6 • Feature D — Plagiarism Checker

SBERT + pgvector

What it does

A student pastes a draft passage; the checker returns an overall risk level (*low/medium/high*), an overall similarity score, and a list of flagged sentence-chunks with the exact matched source paper (title + year) and its similarity score.

Why SBERT + a vector database instead of string matching

Academic plagiarism is most often paraphrasing, not verbatim copying, so exact-match or n-gram techniques alone would miss most real cases. Embedding each chunk and comparing it against the **live, indexed corpus in Supabase** (rather than a static offline index) lets Module 1 catch semantic overlap against the full, continuously-growing set of scraped papers, including ones added after any model was last trained.

How it is implemented (`app/routers/plagiarism.py`)

1. `_chunk_text` splits the input into groups of 3 sentences (regex sentence-boundary split), each chunk padded until >30 characters, so short single-sentence fragments don't get treated as full chunks.
2. Each chunk is embedded with the shared SBERT encoder (`services/shared/embedding_utils.embed` , which lazy-loads `settings.sbert_model_name`).
3. The embedding is sent to the Postgres RPC function `match_papers(query_embedding, match_threshold, match_count)` (defined in `supabase/migrations/008_functions_realtime.sql`), which uses pgvector's cosine-distance operator directly in SQL:

```
SELECT rp.id, rp.title, rp.abstract, 1 - (rp.embedding <=> query_embedding) AS similarity
FROM research_papers rp
WHERE rp.embedding IS NOT NULL AND 1 - (rp.embedding <=> query_embedding) > match_threshold
ORDER BY rp.embedding <=> query_embedding LIMIT match_count;
```

4. The single nearest match (`match_count = 1`) is kept per chunk; if its similarity clears the caller-supplied `threshold` (default 0.8) it is added to `flagged_passages` .

```
overall_score = max(similarity across all chunks) risk_level = "high" if overall_score ≥ 0.90 OR ≥3 chunks flagged
"medium" if overall_score ≥ threshold OR ≥1 chunk flagged "low" otherwise
```

If the shared Supabase/embedding utilities cannot be imported (e.g. local dev without a DB), the endpoint degrades gracefully to `risk_level="low"` with an empty flag list rather than erroring.

7 • Feature E — Mind Map Builder

Gemini concept extraction

term-frequency fallback

What it does

Given a block of text (e.g. an abstract or a set of notes), the Mind Map page returns a hierarchical graph — one central node, and up to `max_nodes-1` concept nodes tiered into *primary/secondary/detail* importance — rendered by the frontend as an interactive D3/force graph.

Why Gemini, with a deterministic fallback

Extracting "the 20 most important concepts and how they relate" from arbitrary free text is a task generative LLMs are well-suited to (it needs real language understanding of importance and hierarchy, not just retrieval). Because this is best-effort/visual rather than something a mark depends on, Module 1 accepts an LLM in the loop here — unlike the gap/proposal/plagiarism features, which stay grounded in the corpus. A pure-local fallback still guarantees the endpoint always returns *something* even with no Gemini quota.

How it is implemented (`app/routers/mindmap.py`)

1. **Gemini path:** a structured prompt asks for a JSON object `{"central": str, "concepts": [{"label", "weight" 0.1-1.0, "related_to_central", "parent"?}]}` over the first 4000 characters of input, parsed via `shared/gemini_client.generate_json` . Each concept becomes a node; edges connect either to its declared `parent` or to the central node.
2. **Local fallback** (`_fallback_mindmap` , triggered on any Gemini exception): regex-extracts words ≥ 4 letters, removes ~ 35 stop-words, and counts frequency with `collections.Counter` . The most frequent term becomes the central node; every node's weight is its count normalised by the top term's count (`count / top_count`); every node gets a direct edge to the centre.
3. **Tier assignment** (`_assign_tiers` , applied to both paths): non-central nodes are sorted by weight and bucketed — top 25% → *primary*, next 35% → *secondary*, remainder → *detail* — so the frontend can colour nodes by importance regardless of which path produced them.

8 • Planned vs. Production Models

The original R26-IT-116 spec called for two deep-learning models for this module. Both exist in the codebase as inference-time wrapper classes but are **not wired into any live router** — the shipped product uses the lighter approaches described above instead:

Spec model	File	Status	What runs in production instead
Citation NER — fine-tuned spaCy <code>en_core_web_trf</code> , entities: AUTHOR/TITLE/JOURNAL/YEAR/VOLUME/PAGES/DOI	<code>app/models/citation_ner.py</code>	Wrapper class present; not imported by any router	Regex-based <code>citation_engine.py</code> (Feature A)
Proposal Generator — Mistral-7B/Llama-2 + RAG (pgvector top-5) + LoRA (r=16, α=32), JSON output	<code>app/models/proposal_generator.py</code>	Wrapper class present (falls back to raw GPT-2 if no LoRA weights found); not imported by any router	SBERT retrieval + templating, <code>proposal_retriever.py</code> (Feature C)

Why this matters for the report: this is a deliberate engineering trade-off, not an oversight — the retrieval-based approaches are faster, cheaper to run, fully explainable, and immune to LLM hallucination, which matters more for an academic-integrity tool than raw generative fluency. Both wrapper classes remain in the codebase and are trainable via `services/module1-integrity/training/` scripts, so they can be swapped in if/when GPU inference becomes available.

9 • Endpoint Reference

Method & Path (via gateway /api/v1/module1/...)	Feature	Backing service
POST /citations/parse	Citation parser	citation_engine.py
POST /citations/format	Citation formatter	citation_engine.py
POST /citations/lookup-doi	DOI lookup	crossref_lookup.py
POST /citations/lookup-title	Title search	crossref_lookup.py
POST /citations/in-text	In-text citation	citation_engine.py
POST /citations/reference-list	Reference list render	citation_engine.py
POST /citations/similar-papers	SBERT similar-paper lookup	sliit_examples.py
POST /gaps/analyze	Gap analysis (topic)	gap_intelligence.py + gap_analyzer.py
POST /gaps/analyze-pdf	Gap analysis (PDF upload)	pdf_extractor.py + gap_intelligence.py
POST /gaps/analyze-full-paper	Gap analysis (pasted text)	pdf_extractor.py + gap_intelligence.py
POST /gaps/report	HTML report download	gap_intelligence.generate_report_html
GET /gaps/status	Model health	gap_analyzer.get_model_info
POST /proposals/generate	Proposal generation	proposal_retriever.py (+ Gemini fallback)
GET /proposals/status	Model health	proposal_retriever.get_model_info
POST /plagiarism/check	Plagiarism check	embedding_utils.py + pgvector RPC
POST /mindmaps/generate	Mind map	Gemini + local term-frequency fallback